

DOCUMENTO DE ALCANCE

Sistema de Gestión de Finanzas Personales

MVP — Versión refinada

1. Descripción del Proyecto

El sistema es una aplicación web de gestión de finanzas personales orientada al ciudadano común sin conocimientos técnicos financieros. Su propósito es centralizar ingresos y gastos, visibilizar hábitos de consumo mediante datos objetivos y permitir la planeación presupuestal para mejorar la salud financiera del usuario.

1.1 Problema que resuelve

Problema	Necesidad de negocio	Impacto esperado
Opacidad financiera	Centralizar información dispersa de ingresos y gastos	Reducción de incertidumbre sobre el paradero del dinero
Falta de planificación	Establecer metas de gasto y límites para evitar sobreendeudamiento	Mejora de la capacidad de ahorro y disciplina financiera
Sesgo cognitivo	Corregir percepción errónea de hábitos de consumo mediante datos objetivos	Decisiones de compra más conscientes y menos emocionales
Desconocimiento técnico	Democratizar herramientas de análisis financiero para el ciudadano común	Mayor inclusión y salud financiera en la población general

2. MVP

El MVP cubre el flujo completo del usuario estándar para registrar, categorizar, presupuestar y monitorear sus finanzas personales. Todo lo que requiera lógica de inteligencia artificial, reportes gráficos avanzados o administración de sistema queda excluido de esta fase.

2.1 Dentro del MVP

- Autenticación completa: registro, inicio de sesión y cierre de sesión
- Gestión de cuentas/bolsillos (CRUD): Efectivo, Banco, Ahorros, etc.
- Gestión de categorías mixtas (predefinidas del sistema + personalizadas por el usuario)
- Registro de transacciones: ingresos y gastos asociados a una cuenta y categoría
- Historial de transacciones con consulta por fecha y categoría
- Búsqueda y filtrado de transacciones
- Creación y configuración de presupuesto (global + por categoría, periodos: mensual, trimestral, semestral, anual)
- Monitoreo y ejecución del presupuesto activo en tiempo real
- Notificación de alerta al alcanzar el 80% del presupuesto

3. Entidades del Dominio

Las siguientes entidades conforman el modelo de dominio del sistema. Sus atributos, tipos y relaciones son la base directa para el diagrama de clases y el diagrama de dominio.

Entidad	Atributos clave	Relaciones principales
Usuario	id, nombre, correo, passwordHash, rol (ENUM), moneda, activo, fechaRegistro, ultimoAcceso	1 Usuario → N Cuentas, N Categorías, 1 Presupuesto activo
Cuenta	id, usuariold, nombre, tipo (ENUM), saldo, activo, fechaCreacion	1 Cuenta → N Transacciones (origen/destino)
Categoria	id, usuariold (null=sistema), nombre, tipo (ENUM), origen (ENUM), icono, activa	1 Categoria → N Transacciones, N PresupuestoCategoria
Transaccion	id, cuentald, categoriald (null en transferencia), tipo (ENUM), monto, fecha, descripcion, cuentaDestinold (null si no es transferencia)	N Transacciones → 1 Cuenta, 1 Categoria, 1 Usuario
Presupuesto	id, usuariold, montoGlobalLimite, periodo (ENUM), fechaInicio, fechaFin, estado (ENUM)	1 Presupuesto → N PresupuestoCategoria
PresupuestoCategoria	id, presupuestold, categoriald, montoLimite, montoEjecutado	N:1 Presupuesto, N:1 Categoria
UsuarioCuenta	usuariold, cuentald, permiso (ENUM: PROPIETARIO COLABORADOR)	Soporte arquitectural para cuentas compartidas en fases posteriores

3.1 Enumeraciones del dominio

Usuario.rol

- USUARIO_ESTANDAR — usuario final, accede solo a sus propios datos
- ADMINISTRADOR — gestiona el sistema, sin acceso a datos financieros ajenos

Cuenta.tipo

- EFECTIVO
- BANCO
- AHORRO
- OTRO — para cuentas personalizadas

Categoría.tipo

- INGRESO — solo asignable a transacciones de tipo ingreso
- EGRESO — solo asignable a transacciones de tipo gasto
- AMBOS — asignable a cualquier tipo de transacción

Categoría.origen

- SISTEMA — predefinida, visible para todos los usuarios
- USUARIO — creada por el usuario, privada

Transacción.tipo

- INGRESO — suma al saldo de la cuenta, no consume presupuesto como gasto
- EGRESO — resta al saldo de la cuenta, consume el presupuesto activo
- TRANSFERENCIA — mueve saldo entre cuentas, no afecta balance ni presupuesto (Fase 2)

Presupuesto.periodo

- MENSUAL — ciclo de 1 mes
- TRIMESTRAL — ciclo de 3 meses
- SEMESTRAL — ciclo de 6 meses
- ANUAL — ciclo de 12 meses

Presupuesto.estado

- BORRADOR — creado pero no activado por el usuario
- ACTIVO — en ejecución, recibe imputación de gastos
- VENCIDO — fecha fin alcanzada, genera BORRADOR del siguiente ciclo
- CERRADO — cancelado manualmente por el usuario antes de vencer

4. Reglas de Negocio

Las siguientes reglas son restricciones del dominio que deben validarse en el backend independientemente de las validaciones del frontend. Son la base para los flujos del BPMN y las precondiciones de los criterios de aceptación de cada HU.

4.1 Reglas de Usuario y Autenticación

Regla 1 — Correo único en el sistema: No pueden existir dos usuarios registrados con el mismo correo electrónico. La validación es case-insensitive.

Regla 2 — Contraseña segura: Mínimo 8 caracteres, al menos una mayúscula y un número. Almacenada siempre con hash (bcrypt, mínimo 10 rondas). Nunca en texto plano.

Regla 3 — Rol no modificable por el usuario: El usuario no puede cambiar su propio rol. La asignación del rol ADMINISTRADOR se realiza directamente en base de datos o por otro administrador.

Regla 4 — Soft delete de usuarios: Un usuario desactivado (activo = false) no puede iniciar sesión pero sus datos se conservan íntegros. El administrador puede reactivarlo.

Regla 5 — Aislamiento total de datos: Toda consulta está filtrada por usuarioid. Un usuario nunca puede ver, editar ni eliminar datos financieros de otro usuario.

4.2 Reglas de Cuentas/Bolsillos

Regla 1 — Cuenta obligatoria en transacción: Toda transacción debe estar asociada a una cuenta. No existen transacciones sin cuenta asignada.

Regla 2 — Cuenta por defecto al registrarse: Al completar el registro, el sistema crea automáticamente una cuenta por defecto (tipo EFECTIVO) para que el usuario pueda operar de inmediato.

Regla 3 — Soft delete con advertencia: Si el usuario intenta eliminar una cuenta con saldo mayor a cero o con transacciones asociadas, el sistema advierte y solicita confirmación explícita. La eliminación marca la cuenta como activa = false.

Regla 4 — Saldo suficiente en transferencia: En una transferencia entre cuentas, el sistema valida que la cuenta origen tenga saldo igual o mayor al monto antes de ejecutar el movimiento.

4.3 Reglas de Categorías

Regla 1 — Sin duplicados por usuario: No pueden existir dos categorías con el mismo nombre para el mismo usuario. La validación es case-insensitive e incluye tanto categorías del sistema como del usuario.

Regla 2 — Soft delete obligatorio: Ninguna categoría con transacciones asociadas se elimina físicamente. Se marca como activa = false para preservar la integridad del historial.

Regla 3 — Al menos una categoría activa: El sistema impide eliminar la última categoría activa de un usuario. Siempre debe existir al menos una para poder registrar transacciones.

Regla 4 — Categorías del sistema editables: El usuario puede editar nombre e icono de las categorías del sistema. El campo origen permanece como SISTEMA. No se pueden restaurar automáticamente si el usuario las elimina.

Regla 5 — Transferencias sin categoría: Las transacciones de tipo TRANSFERENCIA no llevan categoría. El campo categorioid es null por diseño en esos casos.

4.4 Reglas de Transacciones

Regla 1 — Tipo obligatorio: Toda transacción debe tener un tipo explícito: INGRESO, EGRESO o TRANSFERENCIA. No existen transacciones sin tipo.

Regla 2 — Solo egresos consumen presupuesto: Únicamente las transacciones de tipo EGRESO se imputan contra el presupuesto activo. Los ingresos no afectan el consumo presupuestal.

Regla 3 — Monto mayor a cero: El monto de cualquier transacción debe ser un valor numérico positivo mayor a cero.

Regla 4 — Fecha no puede ser futura: El sistema no permite registrar transacciones con fecha posterior a la fecha actual del sistema.

4.5 Reglas de Presupuesto

Regla 1 — Solo un presupuesto ACTIVO por usuario: El sistema impide activar un nuevo presupuesto si ya existe uno en estado ACTIVO. El usuario debe cerrarlo o esperar a que venza.

Regla 2 — fechaFin calculada por el sistema: El usuario elige periodo y fechaInicio. El sistema calcula fechaFin: MENSUAL: +1 mes, TRIMESTRAL: +3 meses, SEMESTRAL: +6 meses, ANUAL: +12 meses.

Regla 3 — Límites por categoría no superan el global: Si existe un montoGlobalLimite, la suma de todos los límites por categoría no puede superarlo. El sistema valida esto al guardar y muestra el disponible en tiempo real.

Regla 4 — Reinicio automático controlado al vencer: Al alcanzar fechaFin, el sistema marca el presupuesto como VENCIDO y crea uno nuevo en estado BORRADOR con el mismo periodo y límites como plantilla. El usuario decide si lo activa o modifica.

Regla 5 — Alerta al 80% del límite: El sistema notifica al usuario cuando el gasto ejecutado alcanza el 80% del montoGlobalLimite o del límite de cualquier categoría individual en el presupuesto activo.

Regla 6 — Historial conservado: Los presupuestos VENCIDOS y CERRADOS se conservan con su ejecución real para consulta histórica. No se eliminan al crear un nuevo ciclo.

5. BPMN

5.1 Flujo de Registro de Usuario

- Actor: Usuario nuevo
- Disparador: El usuario accede a la pantalla de registro
- Pasos:
 - 1. Usuario ingresa nombre, correo y contraseña.
 - 2. Sistema valida formato de correo y requisitos de contraseña.
 - 3. Decisión: ¿El correo ya existe? → Sí: muestra error. No: continúa.
 - 4. Sistema hashea la contraseña y crea el usuario con rol USUARIO_ESTANDAR
 - 5. Sistema crea cuenta por defecto (EFECTIVO) asociada al nuevo usuario
 - 6. Sistema redirige al usuario a la pantalla de inicio
- Resultado: Usuario registrado con cuenta por defecto activa

5.2 Flujo de Inicio y Cierre de Sesión

- Actor: Usuario registrado
- Inicio de sesión:
 - 1. Usuario ingresa correo y contraseña
 - 2. Sistema valida credenciales contra el hash almacenado
 - 3. Decisión: ¿Credenciales válidas y usuario activo? → No: muestra error. Sí: continúa
 - 4. Sistema genera token de sesión y redirige al dashboard
- Cierre de sesión:
 - 1. Usuario presiona 'Cerrar sesión'
 - 2. Sistema invalida el token de sesión
 - 3. Sistema redirige a la pantalla de inicio de sesión

5.3 Flujo de Gestión de Cuentas

- Actor: Usuario autenticado
- Crear cuenta:
 - 1. Usuario ingresa nombre y tipo de cuenta
 - 2. Sistema crea la cuenta con saldo inicial en cero
- Editar cuenta:
 - 1. Usuario selecciona la cuenta y modifica nombre o tipo
 - 2. Sistema actualiza los datos
- Eliminar cuenta:
 - 1. Usuario solicita eliminar
 - 2. [Decisión] ¿Tiene saldo > 0 o transacciones? → Sí: solicita confirmación. No: elimina directamente
 - 3. Sistema marca cuenta como activa = false (soft delete)

5.4 Flujo de Registro de Transacción (Ingreso o Gasto)

- Actor: Usuario autenticado
- Disparador: Usuario accede al formulario de nueva transacción
- Pasos:
 - 1. Usuario selecciona tipo (INGRESO o EGRESO)
 - 2. Usuario selecciona cuenta, categoría, ingresa monto, fecha y descripción opcional
 - 3. Sistema valida: monto > 0, fecha no futura, cuenta activa, categoría activa
 - 4. Decisión si EGRESO: ¿Existe presupuesto ACTIVO? → Sí: imputa el monto al presupuesto y verifica si supera el 80%
 - 5. Decisión: ¿Supera el 80% del límite? → Sí: genera notificación de alerta
 - 6. Sistema actualiza el saldo de la cuenta y guarda la transacción
- Resultado: Transacción registrada, saldo actualizado, presupuesto imputado si aplica

5.5 Flujo de Gestión de Presupuesto

- Actor: Usuario autenticado
- Crear y configurar presupuesto:
 - 1. Decisión: ¿Ya existe un presupuesto ACTIVO? → Sí: bloquea la creación. No: continúa
 - 2. Usuario elige periodo, fecha de inicio y monto global (opcional).
 - 3. Usuario asigna límites por categoría (opcionales)
 - 4. Decisión: ¿Suma de límites por categoría supera el global? → Sí: muestra error. No: guarda como BORRADOR
 - 5. Usuario activa el presupuesto → estado cambia a ACTIVO
- Monitoreo:
 - 1. Cada vez que se registra un EGRESO, el sistema recalcula el monto ejecutado
 - 2. Si ejecutado >= 80% del límite global o por categoría → genera notificación
- Vencimiento automático:
 - 1. Al llegar la fechaFin, sistema marca presupuesto como VENCIDO
 - 2. Sistema crea un nuevo BORRADOR con los mismos parámetros como plantilla

5.6 Flujo de Consulta de Historial y Búsqueda

- Actor: Usuario autenticado
- Historial:
 - 1. Sistema muestra listado de transacciones ordenadas por fecha descendente
 - 2. Usuario puede filtrar por cuenta, tipo, categoría o rango de fechas
- Búsqueda:
 - 1. Usuario ingresa texto libre (descripción) o parámetros específicos
 - 2. Sistema retorna las transacciones que coinciden con los criterios

6. Backlog del MVP

ID	Historia de Usuario	Prioridad	Sprint
HU-01	Registro de usuario	Alta	S1
HU-02	Inicio de sesión	Alta	S1
HU-03	Cerrar sesión	Alta	S1
HU-04	Gestión de cuentas/bolsillos (CRUD) — Nueva	Alta	S1
HU-05	Gestión de categorías (CRUD + reglas)	Alta	S1
HU-06	Registro de ingreso	Alta	S2
HU-07	Registro de gasto	Alta	S2
HU-08	Historial de transacciones	Alta	S2
HU-09	Creación y configuración de presupuesto	Alta	S2
HU-10	Monitoreo y ejecución del presupuesto activo	Alta	S3
HU-11	Notificación de límite de presupuesto (80%)	Media	S3
HU-12	Búsqueda y filtrado de transacciones	Media	S3

6.1 Meta de cada sprint

- Sprint 1 (11 pts): El usuario puede registrarse, autenticarse y configurar su entorno base (cuentas y categorías). Sistema con identidad propia.
- Sprint 2 (12 pts): El usuario registra ingresos y gastos, consulta su historial y configura su presupuesto. Producto funcionalmente usable.
- Sprint 3 (10 pts): El usuario monitorea su presupuesto, recibe alertas antes de excederse y filtra sus transacciones. Producto completo y controlado.

7. Restricciones y Decisiones Arquitecturales

Las siguientes decisiones de diseño fueron tomadas durante el refinamiento del alcance y deben respetarse en la arquitectura del sistema para garantizar coherencia con el modelo de dominio y facilitar la evolución del mismo en caso de ser requerido para fases posteriores.

7.1 Soft delete generalizado

Usuarios, cuentas y categorías no se eliminan físicamente de la base de datos. Se marcan con `activo = false`. Esto preserva la integridad referencial del historial de transacciones y permite recuperar datos en caso de eliminación accidental.

7.2 Tabla UsuarioCuenta para escalabilidad futura

Aunque en el MVP una cuenta pertenece a un único usuario (relación directa), se introduce desde ahora una tabla intermedia UsuarioCuenta con campos `usuariold`, `cuentald` y `permiso` (ENUM: PROPIETARIO | COLABORADOR). En el MVP solo existirá el rol PROPIETARIO, pero la estructura permitirá implementar cuentas compartidas en fase posterior sin reescritura del modelo de datos.

7.3 Validación en backend obligatoria

Todas las reglas de negocio del Capítulo 4 deben validarse en el servidor, independientemente de que también existan validaciones en el frontend. Las validaciones del cliente son para experiencia de usuario; las del servidor son para integridad del sistema.

7.4 Seguridad de contraseñas

Las contraseñas se almacenan exclusivamente con hash bcrypt con un mínimo de 10 rondas de sal. Nunca deben persistirse, loguearse ni transmitirse en texto plano bajo ninguna circunstancia.

7.5 Moneda configurable por usuario

El campo moneda en la entidad Usuario almacena el código ISO 4217 (ej: COP, USD, EUR). Todos los montos se almacenan como valores numéricos sin símbolo de moneda. La presentación del símbolo es responsabilidad del frontend según la moneda del usuario.

8. Glosario del Dominio

Término	Definición
Bolsillo / Cuenta	Contenedor de dinero del usuario. Representa una fuente real: efectivo físico, cuenta bancaria, cuenta de ahorros, etc.
Ingreso	Transacción que incrementa el saldo de una cuenta. Ej: salario, venta, transferencia recibida.
Egreso / Gasto	Transacción que disminuye el saldo de una cuenta y se imputa contra el presupuesto activo. Ej: compra, pago de factura.
Transferencia	Movimiento de dinero entre dos cuentas del mismo usuario. No altera el patrimonio neto ni consume presupuesto. (Fase 2.)
Presupuesto	Límite de gasto definido por el usuario para un periodo determinado (mensual, trimestral, semestral o anual). Puede tener un límite global y límites individuales por categoría.
Patrimonio neto	Diferencia entre todos los ingresos y todos los egresos del usuario desde su registro. Representa el saldo real disponible.
Soft delete	Estrategia de eliminación lógica: el registro se marca como inactivo pero no se borra físicamente de la base de datos.
DoR	Definition of Ready: conjunto de condiciones que debe cumplir una HU antes de poder ser trabajada en un sprint.
DoD	Definition of Done: conjunto de criterios que deben cumplirse para considerar una HU completamente terminada.
INVEST	Principios de calidad para historias de usuario: Independiente, Negociable, Valiosa, Estimable, Small (pequeña), Testeable.

9. Historias de Usuario

HU-01 · Registro de Usuario

Sprint 1 · Prioridad: Alta

Como usuario nuevo sin cuenta en el sistema,

Quiero registrarme con mi nombre, correo y contraseña,

Para poder acceder al sistema y comenzar a gestionar mis finanzas personales.

DoR — Definition of Ready

- Campos del formulario definidos: nombre, correo electrónico, contraseña
- Reglas de validación acordadas y documentadas (Reglas 4.1.1 y 4.1.2)
- Comportamiento ante correo duplicado definido (Regla 4.1.1)
- Mecanismo de hash de contraseña definido: bcrypt mínimo 10 rondas (Regla 4.1.2 y 7.4)
- Flujo de creación de cuenta por defecto al registrarse definido (Regla 4.2.2)
- Diseño del formulario de registro disponible o aprobado.

DoD — Definition of Done

- Formulario implementado con los tres campos requeridos: nombre, correo, contraseña
- Validación de formato de correo aplicada en frontend y backend
- Validación de contraseña aplicada en frontend y backend: mínimo 8 caracteres, al menos una mayúscula y un número
- Contraseña almacenada únicamente con hash bcrypt — nunca en texto plano
- Sistema rechaza registro si el correo ya existe (case-insensitive)
- Al completar el registro, el sistema crea automáticamente una cuenta por defecto de tipo EFECTIVO asociada al nuevo usuario
- Usuario creado con rol USUARIO_ESTANDAR por defecto
- Pruebas unitarias sobre validaciones escritas y pasando
- Prueba de integración del flujo completo exitosa
- Código revisado y aprobado en pull request.

Criterios de Aceptación

Escenario 1: Registro exitoso con datos válidos

- Given que soy un usuario sin cuenta registrada en el sistema
- And me encuentro en el formulario de registro
- When ingreso un nombre válido, un correo con formato correcto
- And ingreso una contraseña con mínimo 8 caracteres, una mayúscula y un número
- And presiono "Registrarse"
- Then el sistema crea mi cuenta con rol USUARIO_ESTANDAR
- And el sistema crea automáticamente una cuenta de tipo EFECTIVO asociada a mi usuario
- And soy redirigido a la pantalla de inicio
- And veo una confirmación visual de registro exitoso

Escenario 2: Intento de registro con correo ya existente

- Given que existe un usuario registrado con el correo "usuario@correo.com"
- And me encuentro en el formulario de registro
- When ingreso el correo "USUARIO@correo.com" con datos válidos en los demás campos
- And presiono "Registrarse"
- Then el sistema no crea una cuenta nueva
- And muestra el mensaje: "Este correo ya está registrado"
- And permanezco en el formulario de registro
-

Escenario 3: Contraseña que no cumple los requisitos mínimos

- Given que me encuentro en el formulario de registro
- When ingreso una contraseña que no cumple alguno de los requisitos
- And presiono "Registrarse"
- Then el sistema no envía el formulario
- And muestra un mensaje indicando el requisito específico que no se cumple

Escenario 4: Correo con formato inválido

- Given que me encuentro en el formulario de registro
- When ingreso un correo sin formato válido como "usuariocorreo.com" o "usuario@"
- And presiono "Registrarse"
- Then el sistema no envía el formulario
- And muestra el mensaje: "Ingresa un correo electrónico válido"

Escenario 5: Campo obligatorio vacío

- Given que me encuentro en el formulario de registro
- When dejo uno o más campos obligatorios sin completar
- And presiono "Registrarse"
- Then el sistema no envía el formulario
- And señala visualmente cada campo vacío que es requerido
- Escenario 6: Verificación de cuenta por defecto creada
- Given que completo el registro con datos válidos exitosamente

- When accedo a la sección de cuentas por primera vez
- Then veo una cuenta de tipo EFECTIVO creada automáticamente
- And el saldo de esa cuenta es 0

Notas técnicas

- La validación de correo duplicado debe ser **case-insensitive** en el backend (Regla 4.1.1). Almacenar el correo en minúsculas al guardar.
- La contraseña nunca debe viajar ni almacenarse en texto plano. Hash bcrypt con mínimo 10 rondas de sal (Reglas 4.1.2 y 7.4).
- La creación de la cuenta EFECTIVO por defecto debe ocurrir en la **misma transacción de base de datos** que la creación del usuario para garantizar atomicidad (Regla 4.2.2).
- El rol USUARIO_ESTANDAR se asigna automáticamente — el usuario no elige su rol en el registro (Regla 4.1.3).

HU-02 · Inicio de Sesión

Sprint 1 · Prioridad: Alta · 2 puntos · Depende de: HU-01

Como usuario registrado en el sistema,

Quiero iniciar sesión con mi correo y contraseña,

Para acceder a mi cuenta y gestionar mis finanzas personales.

DoR — Definition of Ready

- HU-01 completada y en producción del entorno de desarrollo
- Mecanismo de autenticación definido: JWT con Spring Security
- Tiempo de expiración del token definido por el equipo
- Comportamiento ante credenciales inválidas definido (Regla 4.1.2)
- Comportamiento ante usuario inactivo definido (Regla 4.1.4)
- Diseño de la pantalla de login disponible o aprobado

DoD — Definition of Done

- Endpoint de login implementado y funcional (`POST /auth/login`)
- Validación de credenciales contra hash bcrypt almacenado
- Sistema genera token JWT al autenticar correctamente
- Sistema rechaza el acceso si el usuario está inactivo (`activo = false`)
- Mensaje de error genérico ante credenciales inválidas — no especifica si el error es en correo o contraseña
- Token JWT retornado al cliente y almacenado correctamente
- Rutas protegidas inaccesibles sin token válido
- Pruebas unitarias del servicio de autenticación escritas y pasando
- Prueba de integración del flujo completo exitosa
- Código revisado y aprobado en pull request

Criterios de Aceptación

Escenario 1: Inicio de sesión exitoso con credenciales válidas

- Given que soy un usuario registrado con correo "usuario@correo.com"
- And mi cuenta está activa (activo = true)
- And me encuentro en la pantalla de inicio de sesión
- When ingreso mi correo y mi contraseña correcta
- And presiono "Iniciar sesión"
- Then el sistema valida mis credenciales contra el hash almacenado
- And genera un token JWT
- And soy redirigido al dashboard principal
- And tengo acceso a las funcionalidades protegidas del sistema

Escenario 2: Credenciales incorrectas

- Given que me encuentro en la pantalla de inicio de sesión
- When ingreso un correo o contraseña incorrectos
- And presiono "Iniciar sesión"
- Then el sistema no genera token ni concede acceso
- And muestra el mensaje genérico: "Correo o contraseña incorrectos"
- And permanezco en la pantalla de inicio de sesión

Escenario 3: Intento de acceso con usuario inactivo

- Given que existe un usuario con correo "usuario@correo.com"
- And ese usuario tiene activo = false
- And me encuentro en la pantalla de inicio de sesión
- When ingreso las credenciales correctas de ese usuario
- And presiono "Iniciar sesión"
- Then el sistema no genera token ni concede acceso
- And muestra el mensaje: "Tu cuenta está desactivada. Contacta al administrador"
- And permanezco en la pantalla de inicio de sesión

Escenario 4: Campos vacíos en el formulario

- Given que me encuentro en la pantalla de inicio de sesión
- When dejo el campo correo o contraseña vacío
- And presiono "Iniciar sesión"
- Then el sistema no envía la petición al servidor
- And señala visualmente los campos vacíos requeridos

Escenario 5: Intento de acceso a ruta protegida sin sesión activa

- Given que no tengo una sesión activa ni token JWT válido
- When intento acceder directamente a una ruta protegida del sistema
- Then el sistema bloquea el acceso
- And me redirige a la pantalla de inicio de sesión
-

Notas técnicas

- Implementar con **Spring Security** + **JWT** (`io.jsonwebtoken:jjwt`). El filtro `JwtAuthenticationFilter` intercepta cada petición y valida el token antes de permitir el acceso.
- El endpoint `POST /auth/login` es público (`.permitAll()` en `SecurityFilterChain`). Todas las demás rutas requieren token válido (`.authenticated()`).
- Usar `BCryptPasswordEncoder.matches(rawPassword, encodedPassword)` para comparar nunca desencriptar.
- El mensaje de error ante credenciales inválidas debe ser **genérico** (Escenario 2): no indicar si el error es el correo o la contraseña — esto previene enumeración de usuarios.
- La verificación de `activo = false` debe hacerse **antes** de retornar el token, idealmente implementando `UserDetailsService` y lanzando `DisabledException` de Spring Security.
- El tiempo de expiración del JWT debe definirse como propiedad en `application.properties` (`jwt.expiration=86400000` para 24h) — no hardcodeado.

HU-03 · Cerrar Sesión

Sprint 1 · Prioridad: Alta

Como usuario autenticado en el sistema,

Quiero cerrar mi sesión activa,

Para proteger mi información financiera cuando termino de usar la aplicación.

DoR — Definition of Ready

- Estrategia de invalidación de token definida por el equipo (blacklist en memoria o en BD)
- Comportamiento del frontend al cerrar sesión definido: eliminar token almacenado y redirigir
- Diseño del botón/opción de cierre de sesión disponible en el prototipoDoD — Definition of Done
- Endpoint de logout implementado (`POST /auth/logout`)
- Token JWT invalidado en el servidor al cerrar sesión
- Token eliminado del almacenamiento del cliente
- Usuario redirigido a la pantalla de inicio de sesión
- Cualquier intento de usar el token invalidado es rechazado con 401
- Pruebas unitarias del flujo de logout escritas y pasando
- Código revisado y aprobado en pull request.

Criterios de Aceptación

Escenario 1: Cierre de sesión exitoso

- Given que tengo una sesión activa con token JWT válido
- And me encuentro en cualquier pantalla del sistema
- When presiono la opción "Cerrar sesión"
- Then el sistema invalida el token JWT en el servidor
- And el token es eliminado del almacenamiento del cliente
- And soy redirigido a la pantalla de inicio de sesión
- And ya no tengo acceso a ninguna ruta protegida

Escenario 2: Intento de reutilizar el token después de cerrar sesión

- Given que cerré sesión exitosamente
- And el token JWT de mi sesión anterior fue invalidado
- When intento realizar una petición al servidor usando ese token
- Then el servidor responde con código 401 Unauthorized
- And el acceso es denegado

Escenario 3: Intento de acceso a ruta protegida tras cerrar sesión

- Given que cerré sesión exitosamente
- When intento navegar directamente a una ruta protegida del sistema
- Then el sistema bloquea el acceso
- And soy redirigido a la pantalla de inicio de sesión

Escenario 4: Cierre de sesión con token ya expirado

- Given que tengo un token JWT que ya expiró por tiempo
- When presiono la opción "Cerrar sesión"
- Then el sistema procesa el logout sin generar error al usuario
- And soy redirigido a la pantalla de inicio de sesión

Notas técnicas

- JWT es stateless por naturaleza — el servidor no guarda sesiones. Para invalidar el token antes de su expiración natural se requiere una **blacklist**. Para el MVP, una blacklist en memoria (`Set<String>` en un `@Service` singleton) es suficiente. En producción se reemplazaría por Redis.
- El filtro `JwtAuthenticationFilter` debe verificar en cada petición si el token está en la blacklist **antes** de autenticar al usuario.
- El endpoint `POST /auth/logout` debe ser protegido (requiere token válido) — solo un usuario autenticado puede cerrar su propia sesión.
- El frontend debe eliminar el token de `localStorage` o `sessionStorage` al recibir respuesta exitosa del logout, independientemente de lo que haga el servidor.
- El Escenario 4 previene que el usuario vea un error si su sesión ya expiró y luego presiona cerrar sesión — el sistema debe manejarlo con gracia.

HU-04 · Gestión de Cuentas/Bolsillos

Sprint 1 · Prioridad: Alta

Como usuario autenticado en el sistema,

Quiero crear, editar y eliminar mis cuentas/bolsillos,

Para organizar mi dinero por fuente y poder asociar mis transacciones a cada una.

DoR — Definition of Ready

- HU-01 completada — usuario registrado con cuenta EFECTIVO por defecto creada
- Tipos de cuenta definidos en el dominio: EFECTIVO, BANCO, AHORRO, OTRO (Enum `Cuenta.tipo`)
- Reglas de soft delete definidas (Regla 4.2.3)
- Comportamiento de cuenta por defecto al registrarse definido (Regla 4.2.2)
- Tabla `UsuarioCuenta` con campo `permiso` (ENUM: PROPIETARIO) incluida en el modelo de datos (Decisión arquitectural 7.2)
- Diseño de la pantalla de gestión de cuentas disponible

DoD — Definition of Done

- -CRUD completo de cuentas implementado:
- -`POST /cuentas` — crear cuenta
- `GET /cuentas` — listar cuentas activas del usuario
- `PUT /cuentas/{id}` — editar nombre o tipo
- `DELETE /cuentas/{id}` — soft delete
- Cada cuenta queda asociada al usuario autenticado vía `UsuarioCuenta` con permiso PROPIETARIO
- Saldo inicial de toda cuenta nueva es 0
- Soft delete implementado: cuenta marcada como `activo = false`, no eliminada físicamente
- Sistema advierte y solicita confirmación antes de desactivar cuenta con saldo > 0 o con transacciones asociadas
- Usuario no puede ver ni operar cuentas de otro usuario (Regla 4.1.5)
- Pruebas unitarias del servicio de cuentas escritas y pasando
- Prueba de integración del CRUD completo exitosa
- Código revisado y aprobado en pull request

Criterios de Aceptación

Escenario 1: Creación exitosa de una cuenta nueva

- Given que soy un usuario autenticado
- And me encuentro en la sección de gestión de cuentas
- When ingreso un nombre de cuenta y selecciono un tipo válido (EFECTIVO, BANCO, AHORRO u OTRO).
- And presiono "Crear cuenta"
- Then el sistema crea la cuenta con saldo inicial de 0
- And la cuenta queda asociada a mi usuario con permiso PROPIETARIO
- And la nueva cuenta aparece en mi listado de cuentas activas

Escenario 2: Edición exitosa de una cuenta

- Given que soy un usuario autenticado
- And tengo al menos una cuenta activa en mi listado
- When selecciono una cuenta y modifico su nombre o tipo
- And presiono "Guardar"
- Then el sistema actualiza los datos de la cuenta
- And veo los cambios reflejados en mi listado de cuentas

Escenario 3: Eliminación de cuenta sin saldo ni transacciones

- Given que soy un usuario autenticado
- And tengo una cuenta activa con saldo = 0 y sin transacciones asociadas
- When solicito eliminar esa cuenta
- Then el sistema la desactiva directamente (activo = false)
- And la cuenta desaparece de mi listado de cuentas activas
- And no se elimina físicamente de la base de datos

Escenario 4: Intento de eliminar cuenta con saldo mayor a cero

- Given que soy un usuario autenticado
- And tengo una cuenta activa con saldo > 0
- When solicito eliminar esa cuenta
- Then el sistema muestra una advertencia indicando que la cuenta tiene saldo
- And solicita confirmación explícita antes de proceder
- When confirmo la eliminación
- Then el sistema marca la cuenta como activo = false
- And la cuenta desaparece de mi listado de cuentas activas

Escenario 5: Intento de eliminar cuenta con transacciones asociadas

- Given que soy un usuario autenticado
- And tengo una cuenta activa con transacciones registradas
- When solicito eliminar esa cuenta
- Then el sistema muestra una advertencia indicando que la cuenta tiene transacciones
- And solicita confirmación explícita antes de proceder
- When confirmo la eliminación
- Then el sistema marca la cuenta como activo = false
- And las transacciones asociadas se conservan íntegras en el historial

Escenario 6: Cuenta por defecto creada automáticamente al registrarse

- Given que completé mi registro exitosamente
- When accedo por primera vez a la sección de cuentas
- Then veo una cuenta de tipo EFECTIVO creada automáticamente por el sistema
- And el saldo de esa cuenta es 0
- And puedo editarla o eliminarla como cualquier otra cuenta

Escenario 7: Aislamiento — usuario no puede ver cuentas de otro usuario

- Given que soy un usuario autenticado
- When consulto el listado de mis cuentas
- Then solo veo las cuentas asociadas a mi propio usuariold
- And no tengo acceso a cuentas de otros usuarios bajo ninguna circunstancia

Notas técnicas

- Todos los endpoints de `/cuentas` deben estar protegidos con JWT. El `usuariold` se extrae del token — **nunca** del cuerpo de la petición — para garantizar el aislamiento de datos (Regla 4.1.5).
- La tabla `UsuarioCuenta` actúa como la relación entre `Usuario` y `Cuenta`. Al crear una cuenta, se inserta automáticamente un registro en `UsuarioCuenta` con `permiso = PROPIETARIO`. Esto soporta la decisión arquitectural 7.2 sin cambios futuros en el modelo.
- El soft delete se implementa con un campo `activo = true/false` en la entidad `Cuenta`. Los endpoints `GET /cuentas` deben filtrar siempre por `activo = true` — nunca retornar cuentas inactivas al usuario.

- La advertencia previa a la eliminación (Escenarios 4 y 5) puede implementarse con un endpoint previo `GET /cuentas/{id}/estado-eliminacion` que retorne si la cuenta tiene saldo o transacciones, para que el frontend muestre la confirmación apropiada.
- Usar `@Transactional` en el servicio para garantizar atomicidad en la creación de cuenta + registro en `UsuarioCuenta`

HU-05 · Gestión de Categorías

Sprint 1 · Prioridad: Alta

Como usuario autenticado en el sistema,

Quiero crear, editar y eliminar mis categorías,

Para clasificar las transacciones y poder analizar mis hábitos financieros por tipo de gasto o ingreso.

DoR — Definition of Ready

- HU-01 completada — usuario autenticado disponible
- Enumeraciones del dominio definidas: `Categoria.tipo` (INGRESO, EGRESO, AMBOS) y `Categoria.origen` (SISTEMA, USUARIO)
- Listado de categorías predefinidas del sistema acordado por el equipo
- Reglas de categorías definidas y documentadas (Reglas 4.3.1 a 4.3.5)
- Comportamiento de soft delete definido (Regla 4.3.2)
- Regla de mínimo una categoría activa definida (Regla 4.3.3)
- Diseño de la pantalla de gestión de categorías disponible

DoD — Definition of Done

- CRUD completo de categorías implementado:
- `POST /categorias` — crear categoría personalizada
- `GET /categorias` — listar categorías activas del usuario (sistema + propias)
- `PUT /categorias/{id}` — editar nombre e icono
- `DELETE /categorias/{id}` — soft delete con validaciones
- Categorías del sistema visibles para todos los usuarios al registrarse
- Categorías personalizadas vinculadas al `usuariold` del usuario autenticado

- Sistema impide duplicados de nombre por usuario (case-insensitive) (Regla 4.3.1)
- Soft delete implementado para categorías con transacciones asociadas (Regla 4.3.2)
- Sistema impide eliminar la última categoría activa del usuario (Regla 4.3.3)
- Usuario puede editar nombre e icono de categorías del sistema — campo `origen` permanece como SISTEMA (Regla 4.3.4)
- Usuario no puede ver ni operar categorías de otro usuario (Regla 4.1.5)
- Pruebas unitarias del servicio de categorías escritas y pasando
- Prueba de integración del CRUD completo exitosa
- Código revisado y aprobado en pull request

Criterios de Aceptación

Escenario 1: Visualización de categorías al ingresar por primera vez

- **Given** que soy un usuario recién registrado
- **When** accedo a la sección de categorías por primera vez
- **Then** veo el listado de categorías predefinidas del sistema
- **And** todas tienen origen = SISTEMA
- **And** están disponibles para clasificar mis transacciones

Escenario 2: Creación exitosa de categoría personalizada

- **Given** que soy un usuario autenticado
- **And** me encuentro en la sección de gestión de categorías
- **When** ingreso un nombre de categoría que no existe en mi listado
- **And** selecciono un tipo válido (INGRESO, EGRESO o AMBOS)
- **And** presiono "Crear categoría"
- **Then** el sistema crea la categoría con origen = USUARIO
- **And** la categoría queda vinculada a mi usuariold
- **And** aparece en mi listado de categorías activas

Escenario 3: Intento de crear categoría con nombre duplicado

- **Given** que soy un usuario autenticado
- **And** tengo una categoría activa llamada "Alimentación" en mi listado
- **When** intento crear una nueva categoría con el nombre "alimentación"
- **And** presiono "Crear categoría"
- **Then** el sistema no crea la categoría
- **And** muestra el mensaje: "Ya tienes una categoría con ese nombre"

Escenario 4: Edición de categoría personalizada

- **Given** que soy un usuario autenticado
- **And** tengo una categoría con origen = USUARIO en mi listado
- **When** selecciono esa categoría y modifico su nombre o icono
- **And** presiono "Guardar"
- **Then** el sistema actualiza los datos de la categoría
- **And** veo los cambios reflejados en mi listado

Escenario 5: Edición de categoría del sistema

- **Given** que soy un usuario autenticado
- **And** tengo una categoría con origen = SISTEMA en mi listado
- **When** selecciono esa categoría y modifico su nombre o icono
- **And** presiono "Guardar"
- **Then** el sistema actualiza el nombre e icono de la categoría
- **And** el campo origen permanece como SISTEMA
- **And** la categoría actualizada aparece en mi listado

Escenario 6: Eliminación de categoría sin transacciones asociadas

- **Given** que soy un usuario autenticado
- **And** tengo una categoría activa sin transacciones asociadas
- **And** tengo al menos otra categoría activa además de esta
- **When** solicito eliminar esa categoría
- **Then** el sistema la desactiva (activo = false)
- **And** la categoría desaparece de mi listado de categorías activas
- **And** no se elimina físicamente de la base de datos

Escenario 7: Intento de eliminar categoría con transacciones asociadas

- **Given** que soy un usuario autenticado
- **And** tengo una categoría activa con transacciones registradas
- **And** tengo al menos otra categoría activa además de esta
- **When** solicito eliminar esa categoría
- **Then** el sistema la marca como activo = false (soft delete)
- **And** la categoría desaparece de mi listado de categorías activas
- **And** las transacciones asociadas conservan su referencia a esa categoría en el historial.

Escenario 8: Intento de eliminar la última categoría activa

- **Given** que soy un usuario autenticado
- **And** solo tengo una categoría activa en mi listado
- **When** solicito eliminar esa categoría
- **Then** el sistema rechaza la operación
- **And** muestra el mensaje: "Debes tener al menos una categoría activa para poder registrar transacciones".
- **And** la categoría permanece activa en mi listado

Escenario 9: Aislamiento — usuario no puede ver categorías de otro usuario

- **Given** que soy un usuario autenticado
- **When** consulto mi listado de categorías
- **Then** veo únicamente las categorías del sistema
- **And** las categorías personalizadas vinculadas a mi propio usuariold
- **And** no tengo acceso a categorías personalizadas de otros usuarios

Notas técnicas

- El `GET /categorias` debe retornar la unión de dos conjuntos: categorías con `usuariold = null` (del sistema) y categorías con `usuariold = {id del usuario autenticado}`. Implementar con una query JPQL: `WHERE c.activa = true AND (c.usuariold IS NULL OR c.usuariold = :usuariold)`.
- La validación de duplicados (Regla 4.3.1) debe hacerse en el backend sobre ese mismo conjunto unificado, aplicando `LOWER(nombre)` para la comparación case-insensitive.
- El `usuariold` para categorías del sistema se almacena como `null` en base de datos — esto distingue las categorías globales de las personalizadas sin necesidad de un campo adicional.
- La Regla 4.3.3 (mínimo una categoría activa) debe validarse en el servicio antes de ejecutar el soft delete: `COUNT(categorias WHERE activa = true AND (usuariold IS NULL OR usuariold = :id)) > 1`.
- Para la edición de categorías del sistema (Escenario 5), el campo `origen` debe ser inmutable — usar `@Column(updatable = false)` en la entidad para garantizarlo a nivel de JPA.
- Todos los endpoints protegidos con JWT. El `usuariold` siempre se extrae del token, nunca del cuerpo de la petición.

HU-06 · Registro de Ingreso

Sprint 2 · Prioridad: Alta

Como usuario autenticado en el sistema

Quiero registrar un ingreso asociado a una cuenta y categoría

Para llevar el control del dinero que entra y ver reflejado el incremento en mi saldo

DoR — Definition of Ready

- HU-00 completada — cuentas activas disponibles para el usuario
- HU-07 completada — categorías activas disponibles para el usuario
- Enumeración Transacción.tipo definida: INGRESO, EGRESO, TRANSFERENCIA
- Reglas de transacciones definidas y documentadas (Reglas 4.4.1 a 4.4.4)
- Campos del formulario definidos: tipo, cuenta, categoría, monto, fecha, descripción (opcional)
- Comportamiento del saldo de cuenta tras registrar ingreso definido
- Diseño del formulario de registro de transacción disponible

DoD — Definition of Done

- Endpoint implementado: POST /transacciones
- Transacción creada con tipo = INGRESO
- Saldo de la cuenta seleccionada incrementado en el monto registrado dentro de la misma transacción @Transactional
- Validaciones aplicadas: monto > 0, fecha no futura, cuenta activa, categoría activa y perteneciente al usuario
- Categorías de tipo EGRESO excluidas del selector al registrar un ingreso
- Transacción almacenada con referencia al usuarioId extraído del token JWT
- Usuario no puede registrar transacciones sobre cuentas de otro usuario
- Pruebas unitarias del servicio de transacciones para ingreso escritas y pasando
- Prueba de integración del flujo completo exitosa
- Código revisado y aprobado en pull request

Criterios de Aceptación

Escenario 1: Registro exitoso de un ingreso

- Given que soy un usuario autenticado
- And tengo al menos una cuenta activa
- And tengo al menos una categoría activa de tipo INGRESO o AMBOS
- And me encuentro en el formulario de registro de transacción
- When selecciono tipo INGRESO
- And selecciono una cuenta activa de mi listado
- And selecciono una categoría de tipo INGRESO o AMBOS
- And ingreso un monto de 500000 mayor a cero
- And ingreso una fecha igual o anterior a la fecha actual
- And presiono "Guardar"
- Then el sistema registra la transacción con tipo = INGRESO

- And el saldo de la cuenta seleccionada se incrementa en 500000
- And la transacción aparece en mi historial

Escenario 2: Intento de registrar ingreso con monto igual a cero o negativo

- Given que soy un usuario autenticado
- And me encuentro en el formulario de registro de transacción con tipo INGRESO
- When ingreso un monto de 0 o un valor negativo
- And presiono "Guardar"
- Then el sistema no registra la transacción
- And muestra el mensaje: "El monto debe ser mayor a cero"

Escenario 3: Intento de registrar ingreso con fecha futura

- Given que soy un usuario autenticado
- And me encuentro en el formulario de registro de transacción con tipo INGRESO
- When ingreso una fecha posterior a la fecha actual del sistema
- And presiono "Guardar"
- Then el sistema no registra la transacción
- And muestra el mensaje: "La fecha no puede ser posterior a la fecha actual"

Escenario 4: Intento de registrar ingreso sin cuenta seleccionada

- Given que soy un usuario autenticado
- And me encuentro en el formulario de registro de transacción con tipo INGRESO
- When no selecciono ninguna cuenta
- And presiono "Guardar"
- Then el sistema no registra la transacción
- And muestra el mensaje: "Debes seleccionar una cuenta"

Escenario 5: Intento de registrar ingreso sin categoría seleccionada

- Given que soy un usuario autenticado
- And me encuentro en el formulario de registro de transacción con tipo INGRESO
- When no selecciono ninguna categoría
- And presiono "Guardar"
- Then el sistema no registra la transacción
- And muestra el mensaje: "Debes seleccionar una categoría"

Escenario 6: Selector de categorías filtra correctamente por tipo

- Given que soy un usuario autenticado
- And me encuentro en el formulario de registro de transacción
- When selecciono tipo INGRESO

- Then el selector de categorías muestra únicamente categorías de tipo INGRESO o AMBOS
- And no muestra categorías de tipo EGRESO

Escenario 7: Descripción opcional no bloquea el registro

- Given que soy un usuario autenticado
- And me encuentro en el formulario de registro de transacción con tipo INGRESO
- And he completado todos los campos obligatorios correctamente
- When dejo el campo descripción vacío
- And presiono "Guardar"
- Then el sistema registra la transacción exitosamente
- And el campo descripción queda almacenado como null

Escenario 8: Atomicidad — saldo actualizado junto con la transacción

- Given que soy un usuario autenticado
- And tengo una cuenta con saldo de 100000
- When registro un ingreso de 50000 en esa cuenta exitosamente
- Then el saldo de la cuenta refleja 150000
- And la transacción queda registrada en el historial
- And ambos cambios ocurren de forma atómica

Notas técnicas

- El endpoint POST /transacciones recibe el tipo en el cuerpo (tipo: "INGRESO") y sirve tanto para HU-02a como para HU-02b. La lógica de negocio se bifurca en el servicio según el tipo.
- Usar @Transactional en el método del servicio para garantizar que el registro de la transacción y la actualización del saldo de la cuenta ocurran de forma atómica — si alguno falla, se hace rollback de ambos.
- El usuarioid se extrae del token JWT, nunca del cuerpo. Antes de persistir, validar que la cuentaid y la categoriaid pertenecen al usuario autenticado — de lo contrario retornar 403 Forbidden.
- El filtro de categorías por tipo (Escenario 6) puede implementarse en el endpoint GET /categorias?tipo=INGRESO o GET /categorias?tipo=AMBOS, o en el frontend filtrando la lista ya cargada.
- Validar fecha <= LocalDate.now() en el servicio, independientemente de la validación del frontend.
- Usar @Positive de Bean Validation para la validación del monto en el DTO de entrada.

